



پاسخ تمرین دوم

پاسخ سوال اول: تحلیل گرادیان و پیاده‌سازی لایه کانولوشن سفارشی (۲۵ نمره)

بخش اول: روابط ریاضی (۱۰ نمره)

فرض کنید خروجی لایه کانولوشن Y باشد. با توجه به اینکه فیلتر نهایی به صورت $F = W \odot M$ تعریف شده است، خروجی به صورت $Y = X * F$ محاسبه می‌گردد (که در آن $*$ عملگر کانولوشن است). طبق قاعده زنجیره‌ای در پس‌انتشار (Backpropagation)، اگر گرادیان تابع خطا نسبت به فیلتر نهایی را با $\frac{\partial L}{\partial F}$ نشان دهیم، طبق روابط استاندارد کانولوشن داریم:

$$\frac{\partial L}{\partial F} = X * \frac{\partial L}{\partial Y} \quad (۱)$$

حال با توجه به رابطه $F_{i,j} = W_{i,j} \times M_{i,j}$ ، مشتقات جزئی نسبت به W و M به صورت زیر خواهند بود:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial F} \odot M, \quad \frac{\partial L}{\partial M} = \frac{\partial L}{\partial F} \odot W \quad (۲)$$

بخش دوم: پیاده‌سازی با PyTorch (۱۵ نمره)

برای پیاده‌سازی بدون `nn.Conv2d`، از `F.unfold` برای استخراج پچ‌های تصویری و ضرب ماتریسی استفاده می‌کنیم تا محاسبات بهینه‌تر از حلقه‌های تو در تو انجام شود.

```
1 import torch
2 import torch.nn.functional as F
3
4 class CustomMaskedConv2d(torch.nn.Module):
5     def __init__(self, kernel_size=3):
6         super().__init__()
7         self.k = kernel_size
8         # Initialize W and M as learnable parameters
9         self.W = torch.nn.Parameter(torch.randn(1, 1, self.k, self.k))
10        self.M = torch.nn.Parameter(torch.ones(1, 1, self.k, self.k))
11
12    def forward(self, x):
13        batch_size, channels, h, w = x.shape
14        # Create the effective filter
15        effective_filter = self.W * self.M
16
17        # Unfold input to patches: shape (Batch, C*k*k, L)
18        # where L is the number of spatial positions
19        unfolded = F.unfold(x, kernel_size=self.k)
20
21        # Reshape filter to (1, C*k*k)
22        filter_flat = effective_filter.view(1, -1)
```

```

23
24     # Perform matrix multiplication
25     # filter_flat: (1, C*k*k), unfolded: (Batch, C*k*k, L) -> (Batch,
26     1, L)
27     out_flat = torch.matmul(filter_flat, unfolded)
28
29     # Calculate output spatial dimensions
30     out_h = h - self.k + 1
31     out_w = w - self.k + 1
32
33     # Reshape back to image format
34     out = out_flat.view(batch_size, 1, out_h, out_w)
35     return out
36
37 # Synthesized Data
38 torch.manual_seed(42)
39 x = torch.randn(1, 1, 5, 5)
40 target = torch.randn(1, 1, 3, 3) # Output of 5x5 with 3x3 kernel and
41 stride 1 is 3x3
42
43 # Training step
44 model = CustomMaskedConv2d(kernel_size=3)
45 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
46
47 output = model(x)
48 loss = F.mse_loss(output, target)
49
50 optimizer.zero_grad()
51 loss.backward()
52
53 print("Loss:", loss.item())
54 print("Gradient of W (sum):", model.W.grad.sum().item())
55 print("Gradient of M (sum):", model.M.grad.sum().item())

```

پاسخ سوال دوم: معماری Bottleneck و مدیریت منابع (۲۵ نمره)

بخش اول: محاسبه تحلیلی پارامترها (۵ نمره)

ابعاد ورودی ۲۵۶ کانال است. تعداد پارامترهای هر لایه کانولوشن بدون Bias برابر است با: $C_{in} \times C_{out} \times K_h \times K_w$.

• لایه اول (کانولوشن 1×1): $256 \times 64 \times 1 \times 1 = 16,384$

• لایه دوم (کانولوشن 3×3): $64 \times 64 \times 3 \times 3 = 36,864$

• لایه سوم (کانولوشن 1×1): $64 \times 256 \times 1 \times 1 = 16,384$

مجموع پارامترها در حالت اول: $16,384 + 36,864 + 16,384 = 69,632$ پارامتر.

بخش دوم: کاهش منابع و تحلیل ویژگی‌ها (۱۰ نمره)

با تغییر فیلترهای میانی از ۶۴ به ۳۲:

• لایه اول: $256 \times 32 \times 1 \times 1 = 8,192$

• لایه دوم: $32 \times 32 \times 3 \times 3 = 9,216$

• لایه سوم: $32 \times 256 \times 1 \times 1 = 8,192$

مجموع پارامترها در حالت جدید: 25,600 پارامتر. درصد کاهش: $\frac{69632-25600}{69632} \times 100 \approx 63.24\%$
 تاثیر بر میدان دریافت (Receptive Field): هیچ تاثیری ندارد! میدان دریافت فضایی تنها به ابعاد هسته مکانی (در اینجا 3×3) وابسته است که تغییر نکرده است. تاثیر بر ظرفیت مدل: کاهش تعداد کانال‌های میانی منجر به کاهش تنوع ویژگی‌های استخراج شده (Feature Maps) شده و ممکن است توانایی مدل در یادگیری الگوهای پیچیده محدود گردد.

بخش سوم: اثبات برنامه‌نویسی (۱۰ نمره)

```
1 import torch
2 import torch.nn as nn
3
4 def create_bottleneck(in_channels, mid_channels, out_channels):
5     return nn.Sequential(
6         nn.Conv2d(in_channels, mid_channels, kernel_size=1, bias=False),
7         nn.Conv2d(mid_channels, mid_channels, kernel_size=3, padding=1,
8             bias=False),
9         nn.Conv2d(mid_channels, out_channels, kernel_size=1, bias=False)
10    )
11
12 bottleneck_64 = create_bottleneck(256, 64, 256)
13 bottleneck_32 = create_bottleneck(256, 32, 256)
14
15 # Calculate parameters
16 params_64 = sum(p.numel() for p in bottleneck_64.parameters())
17 params_32 = sum(p.numel() for p in bottleneck_32.parameters())
18
19 print(f"Parameters (64 filters): {params_64}") # Should be 69632
20 print(f"Parameters (32 filters): {params_32}") # Should be 25600
21
22 # Verify Shapes
23 dummy_input = torch.randn(1, 256, 14, 14)
24 out_64 = bottleneck_64(dummy_input)
25 out_32 = bottleneck_32(dummy_input)
26
27 print(f"Output shape (64): {out_64.shape}")
28 print(f"Output shape (32): {out_32.shape}")
```

پاسخ سوال سوم: معماری YOLO و معیار Soft Dice Loss (۲۵ نمره)

بخش اول: فرمول‌بندی و توجیه ریاضی (۱۰ نمره)
 در مکان‌یابی اشیاء (Localization)، به جای خطای MSE، می‌توانیم از خطای دایس (Dice Loss) بر مبنای مساحت Bounding Box استفاده کنیم. اگر پیش‌بینی را $P = (x_p, y_p, w_p, h_p)$ و هدف را $T = (x_t, y_t, w_t, h_t)$ در نظر بگیریم، ابتدا مساحت اشتراک (Intersection Area) و مساحت هر یک را حساب می‌کنیم:

$$\text{Score Dice} = \frac{2 \times A_{\text{intersection}}}{A_P + A_T + \epsilon} \quad (۳)$$

$$\mathcal{L}_{\text{Dice}} = 1 - \text{Score Dice} \quad (۴)$$

چرا بهتر از MSE است؟ تابع MSE وابستگی شدیدی به مقیاس دارد. یک خطای ۱۰ پیکسلی برای یک شیء بزرگ شاید بی‌اهمیت باشد، اما برای یک شیء کوچک باعث خطای مطلق چشمگیری می‌شود. در مقابل، Dice Score از آنجا که نسبتی از مساحت اشتراک به کل مساحت‌ها است، به صورت ذاتی به مقیاس نامتغیر (Scale-Invariant) عمل می‌کند و هم برای اشیاء بزرگ و هم کوچک عملکرد پایدارتری دارد.

بخش دوم و سوم: پیاده‌سازی و تست سناریو (۱۵ نمره)

```
1 import torch
2
3 def soft_dice_loss_bbox(pred_box, target_box, eps=1e-6):
4     # Boxes are formatted as [x_center, y_center, w, h]
5     # Convert to [x_min, y_min, x_max, y_max]
6     def to_corners(box):
7         x_min = box[..., 0] - box[..., 2] / 2
8         y_min = box[..., 1] - box[..., 3] / 2
9         x_max = box[..., 0] + box[..., 2] / 2
10        y_max = box[..., 1] + box[..., 3] / 2
11        return x_min, y_min, x_max, y_max
12
13    px_min, py_min, px_max, py_max = to_corners(pred_box)
14    tx_min, ty_min, tx_max, ty_max = to_corners(target_box)
15
16    # Calculate Intersection
17    inter_x_min = torch.max(px_min, tx_min)
18    inter_y_min = torch.max(py_min, ty_min)
19    inter_x_max = torch.min(px_max, tx_max)
20    inter_y_max = torch.min(py_max, ty_max)
21
22    inter_w = torch.clamp(inter_x_max - inter_x_min, min=0)
23    inter_h = torch.clamp(inter_y_max - inter_y_min, min=0)
24    inter_area = inter_w * inter_h
25
26    # Calculate Areas
27    pred_area = pred_box[..., 2] * pred_box[..., 3]
28    target_area = target_box[..., 2] * target_box[..., 3]
29
30    # Calculate Dice
31    dice_score = (2 * inter_area + eps) / (pred_area + target_area + eps)
32    return 1.0 - dice_score.mean()
33
34 # Test Scenario (S=2, B=1, coords: x, y, w, h)
35 # Case A: Perfect match
36 target = torch.tensor([[[[0.5, 0.5, 0.2, 0.2], [0.1, 0.1, 0.1, 0.1]],
37                        [[0.8, 0.8, 0.3, 0.3], [0.9, 0.9, 0.2, 0.2]]]])
38 pred_perfect = target.clone()
39
40 # Case B: No overlap
41 pred_disjoint = torch.tensor([[[[0.1, 0.1, 0.1, 0.1], [0.9, 0.9, 0.1,
42                        0.1]],
43                        [[0.2, 0.2, 0.1, 0.1], [0.3, 0.3, 0.1,
44                        0.1]]]])
```

```

44 loss_perfect = soft_dice_loss_bbox(pred_perfect, target)
45 loss_disjoint = soft_dice_loss_bbox(pred_disjoint, target)
46
47 print("Loss for Perfect Match:", loss_perfect.item()) # Expected: ~0.0
48 print("Loss for No Overlap:", loss_disjoint.item())    # Expected: ~1.0

```

پاسخ سوال چهارم: تلفیق DAE و CAE (۲۵ نمره)

بخش اول: تابع هدف و تحلیل ریاضی (۵ نمره)

تابع هدف جامع ترکیبی از خطای بازسازی ورودی نویزدار (DAE) و جریمه انقباضی بر روی مشتقات فضای پنهان (CAE) است:

$$\mathcal{L}(x, \tilde{x}) = \mathbb{E} [\|x - g(f(\tilde{x}))\|_2^2] + \lambda \|J_f(x)\|_F^2 \quad (5)$$

در این فرمول:

- $\tilde{x}$: نمونه آغشته به نویز (نقش DAE). این ترم مدل را مجبور می‌کند نویزها را نادیده گرفته و توزیع اصلی را بازسازی کند.

- $\|J_f(x)\|_F^2$: نرم فروبنیوس ماتریس ژاکوبین لایه پنهان نسبت به ورودی بدون نویز (نقش CAE). این ترم تضمین می‌کند که فضای پنهان نسبت به تغییرات کوچک در فضای ورودی استوار و غیرحساس (Invariant) باقی بماند.

بخش دوم: محاسبه تحلیلی ژاکوبین (۵ نمره)

برای یک لایه پنهان با فعال‌ساز Sigmoid به فرم $h = \sigma(Wx + b)$ ، مشتق تابع فعال‌ساز برابر است با $h(1 - h)$. در نتیجه، نرم فروبنیوس ماتریس ژاکوبین به صورت تحلیلی و بدون نیاز به Autograd سنگین محاسبه می‌شود:

$$\|J_f(x)\|_F^2 = \sum_i (h_i(1 - h_i))^2 \sum_j W_{i,j}^2 \quad (6)$$

بخش سوم: آموزش و تست روی داده‌های ناهنجار (۱۵ نمره)

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class HybridAutoencoder(nn.Module):
6     def __init__(self, input_dim=20, hidden_dim=10):
7         super().__init__()
8         self.encoder = nn.Linear(input_dim, hidden_dim)
9         self.decoder = nn.Linear(hidden_dim, input_dim)
10        self.sigmoid = nn.Sigmoid()
11
12        def forward(self, x):
13            h = self.sigmoid(self.encoder(x))
14            out = self.decoder(h)
15            return out, h
16
17    def contractive_loss(model, h, lambda_val=1e-4):
18        # Analytical Frobenius norm of Jacobian for Sigmoid
19        # h shape: (Batch, Hidden)
20        W = model.encoder.weight # shape: (Hidden, Input)
21        dh = h * (1.0 - h) # derivative of sigmoid

```

```

22     w_sum = torch.sum(W ** 2, dim=1) # Sum over input dimensions
23     # Contractive penalty
24     penalty = torch.sum((dh ** 2) * w_sum, dim=1).mean()
25     return lambda_val * penalty
26
27 # Data Synthesis: 16 samples of 20-dim Sine wave + slight Gaussian noise
28 torch.manual_seed(10)
29 t = torch.linspace(0, 4*3.14, 20)
30 clean_data = torch.sin(t).repeat(16, 1)
31 noisy_data_train = clean_data + 0.1 * torch.randn(16, 20)
32
33 model = HybridAutoencoder()
34 optimizer = optim.Adam(model.parameters(), lr=0.01)
35 criterion = nn.MSELoss()
36
37 # Training Loop
38 for epoch in range(50):
39     optimizer.zero_grad()
40
41     # Forward pass on noisy data
42     reconstructed, h = model(noisy_data_train)
43
44     # MSE Loss compares to CLEAN data (DAE principle)
45     mse_loss = criterion(reconstructed, clean_data)
46
47     # Contractive Loss (CAE principle)
48     c_loss = contractive_loss(model, h, lambda_val=0.1)
49
50     loss = mse_loss + c_loss
51     loss.backward()
52     optimizer.step()
53
54 print(f"Final Training Loss: {loss.item():.4f}")
55
56 # Anomaly Detection Test
57 # Normal sample (Sine + noise)
58 normal_sample = clean_data[0:1] + 0.1 * torch.randn(1, 20)
59 # Anomalous sample (Pure Uniform Noise)
60 anomalous_sample = torch.rand(1, 20) * 2 - 1
61
62 model.eval()
63 with torch.no_grad():
64     rec_normal, _ = model(normal_sample)
65     rec_anomalous, _ = model(anomalous_sample)
66
67     score_normal = F.mse_loss(rec_normal, normal_sample).item()
68     score_anomalous = F.mse_loss(rec_anomalous, anomalous_sample).item()
69
70 print(f"Anomaly Score (Normal Sample): {score_normal:.4f}")
71 print(f"Anomaly Score (Anomalous Sample): {score_anomalous:.4f}")
72 # Anomalous score should be significantly higher

```